

Высокоуровневое проектирование (HLD) с использованием C4 Model

AI-Архитектор



Проверить, идет ли запись

Меня хорошо видно & слышно?





Иван Четвериков

AI Architect

- Более 5 лет опыта в разработке и архитектуре
- Специалист в разработке и проектировании AI-приложений в Raft

 @istrebitel_1

 istrebitel.3.12@gmail.com

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса



СТАРТ

Стратегический фундамент и
планирование проекта

Проектирование и
документирование архитектуры

Продвинутые
архитектурные
паттерны

Инфраструктура

Качество, интеграции и
безопасность

Стратегия, лидерство и
экономика

Проектная работа

ФИНИШ



Маршрут вебинара

Знакомство

Представления описания
архитектуры

C4 модель представлений

C4 на примере

Инструментарий

Рефлексия

Цели вебинара

К концу занятия вы сможете

1. Применять модель C4 для описания архитектуры системы на разных уровнях абстракции

2. Разрабатывать диаграммы контекста, контейнеров и компонентов для AI-решений

3. Визуализировать и доносить архитектурные решения до технических и бизнес-команд

Смысл

Для чего вам это уметь

Для того чтобы выстраивать общее понятийное пространство для различных команд



**Занимался ли кто-нибудь
из вас высокоуровневым
проектированием?**



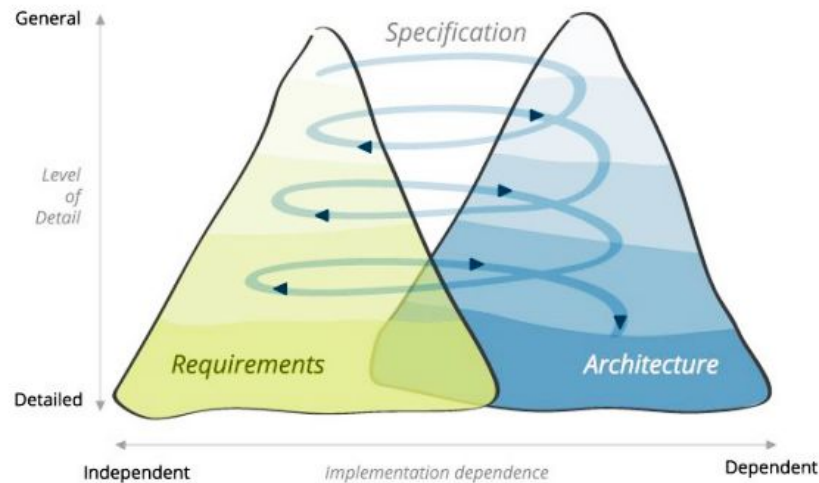
Представления описания архитектуры

**Фактов не существует,
есть только
интерпретации фактов.**

**Фридрих Ницше,
философ**

Главные законы архитектуры

- Суть работы архитектора — Решения, а не Схемы.
 - Диаграммы (C4, UML) — это лишь форма представления.
 - Ценность — это обоснованное техническое решение.
- Архитектура — это функция от Требований.
 - Мы не придумываем систему в вакууме.
- Требования никогда не готовы на 100%.
 - Процесс итеративен

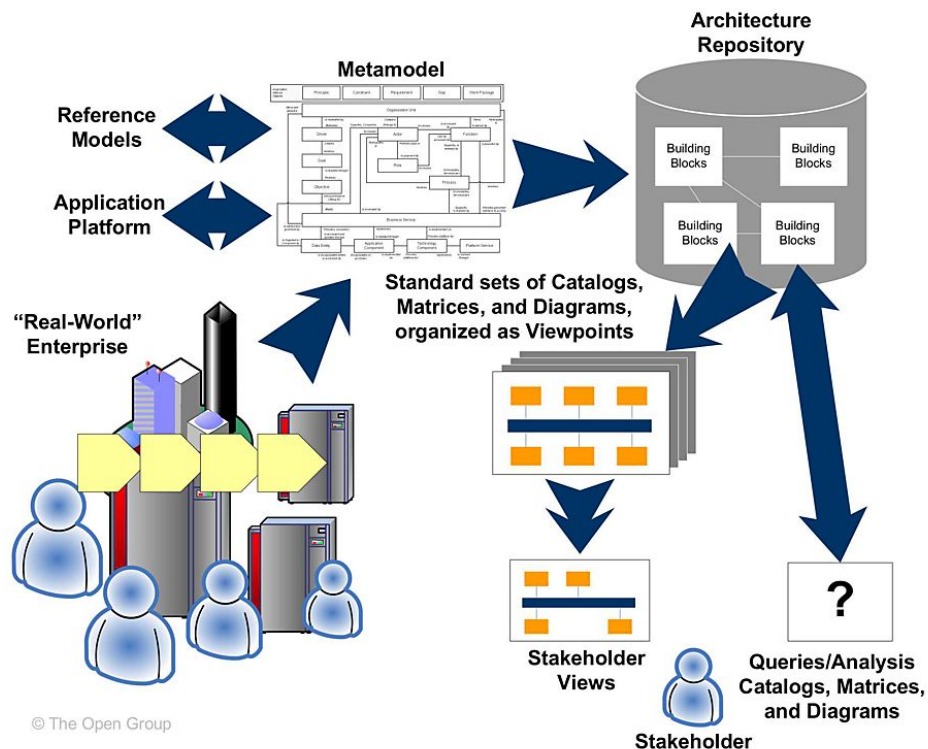


Архитектурные представления в организации

TOGAF – это фреймворк для разработки корпоративной архитектуры.

Ключевая идея построения архитектурных представлений в этом фреймворке - идея контекстной уместности

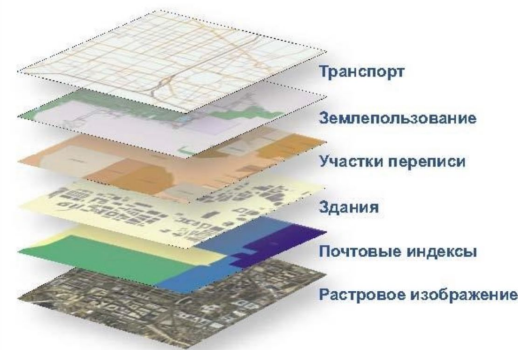
С точки зрения этого фреймворка архитектура строится из блоков, хранящихся в архитектурном репозитории, но каждый адресат видит артефакт, понятный именно ему



Представления описания архитектуры

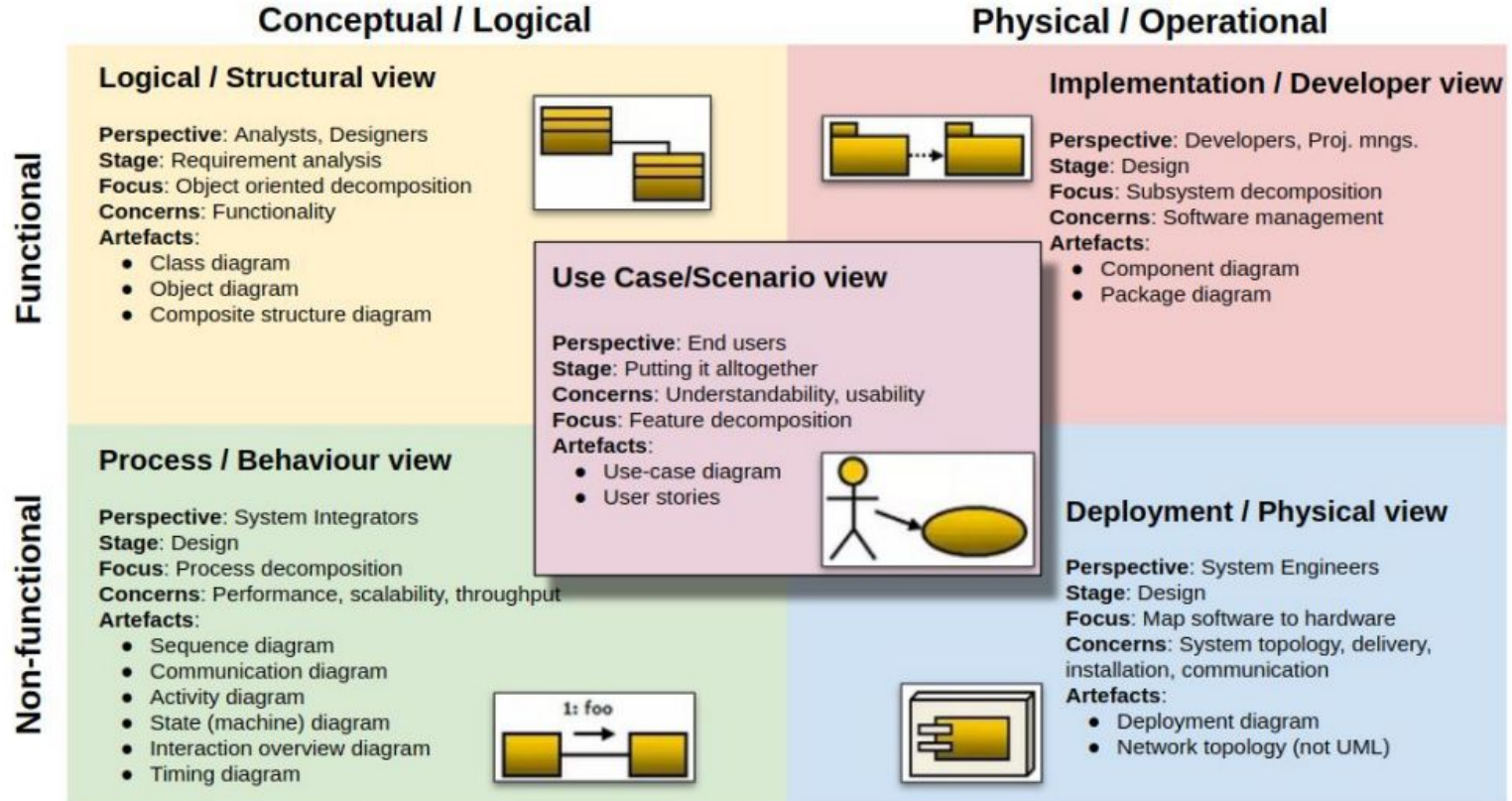
Когда архитектор описывает систему и решение, он описывает ее не максимально подробно, а показывает некоторую «проекцию», взгляд с одной из сторон.

Самая большая проблема при описании архитектур **в смешивании разных взглядов в одном описании.**



ГИС интегрирует многие типы пространственных данных.

Представления описания архитектуры. 4+1



Вопросы



если есть вопросы

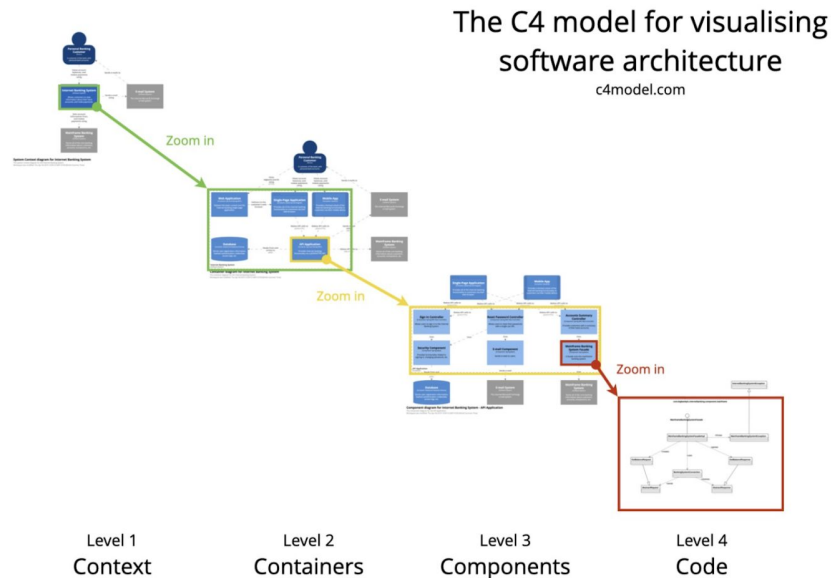


если вопросов нет

C4 модель представлений (Level 1 & 2)

C4 модель Саймона Брауна

Саймон Браун в 2015 году представил модель C4 — идейную наследницу 4+1, но чуть более простую и современную версию. Больше подходящую для описание программной архитектуры.



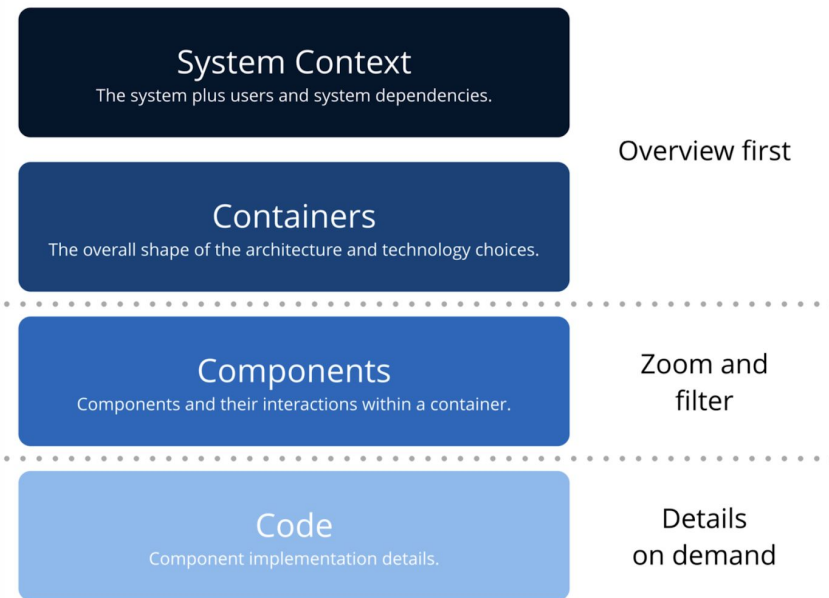
Нотация vs Абстракция

- Любая диаграмма представляет собой набор абстракций, которые между собой каким-то образом связаны, описанных на каком-то языке (нотация).
- Идея Саймона заключалась в том, что не столь важна нотация, а важно про какие абстракции рассказывается на диаграмме.
- В конце концов квадратики и стрелочки можно подписать и добавить легенду.

C4 модель Саймона Брауна

В рамках парадигмы C4 предлагается 4 уровня представлений в архитектуре:

- Контекст
- Контейнер
- Компонент
- Код

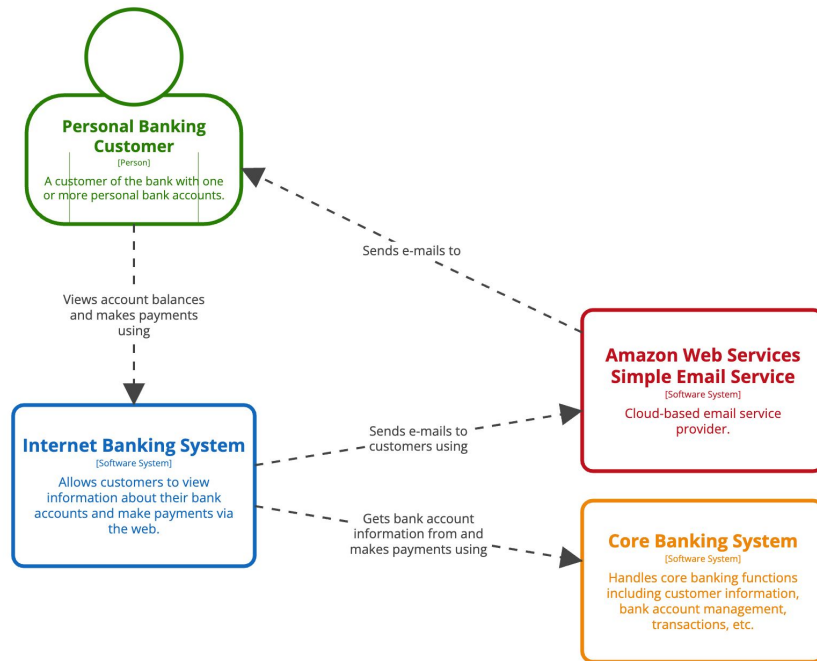


Контекстная диаграмма - это

диаграмма, которая предоставляет первоначальное приближение для системы, показывает как система связана с окружающим миром.

Контекстная диаграмма

- Диаграмма показывает систему в целом и её окружение
- Диаграмма отвечает на следующие вопросы:
 - Про что наше приложение?
 - Кто использует его?
 - Как приложение интегрировано в окружающую среду?
- Основные элементы - система
- Дополнительные элементы - люди и внешние системы
- Целевая аудитория - технические и нетехнические люди внутри команды разработки и вне её



System Context View: Internet Banking System

The system context diagram for a fictional Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

Контейнер - это

что-то, что «запускает» приложение или хранит данные.

Основной критерий: **контейнер** – это что-то, что можно задеплоить независимо друг от друга.

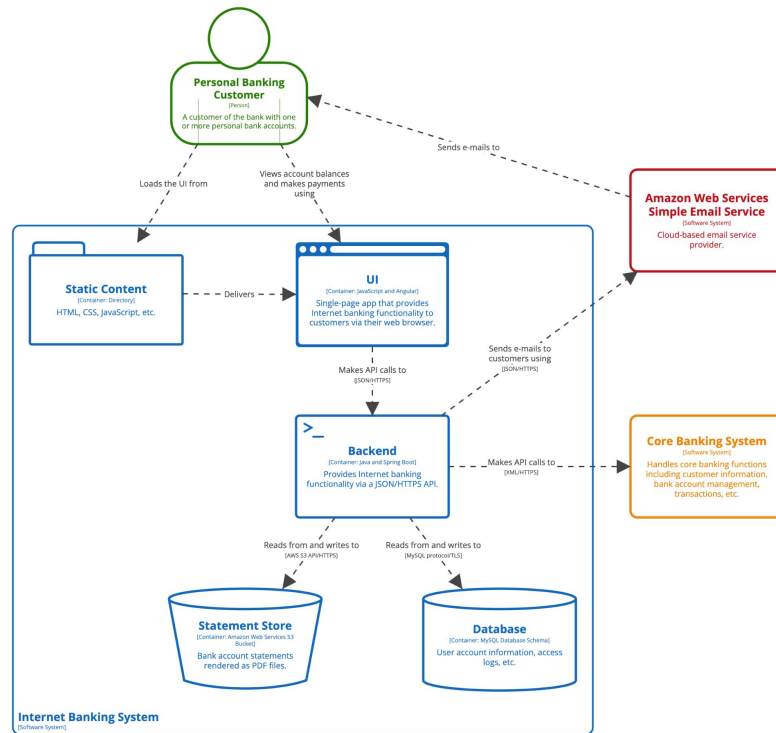
Примеры контейнеров:

- Серверное веб-приложение
- Клиентское веб-приложение
- Мобильное приложение
- Микросервис
- База данных
- Файловая система
- Шелл скрипт
- И т.д.



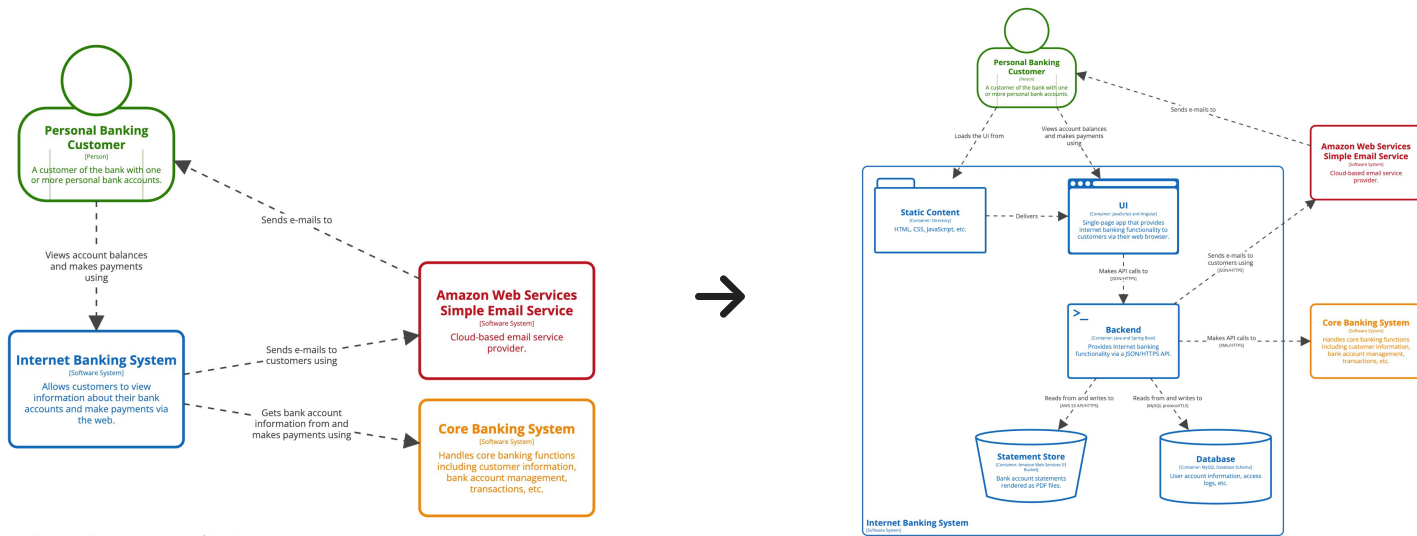
Диаграмма контейнеров

- Диаграмма показывает составные части конкретной системы
- Диаграмма отвечает на следующие вопросы:
 - Из чего состоит приложение?
 - Как контейнеры взаимодействуют друг с другом?
- Основные элементы - система
- Дополнительные элементы - люди и внешние системы непосредственно взаимодействующие с контейнерами
- Целевая аудитория - технический персонал в команде разработки - архитекторы, разработчики, инженеры поддержки



Container View: Internet Banking System
The container diagram for the Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

Взаимосвязь диаграмм контекста и контейнеров



System Context View: Internet Banking System

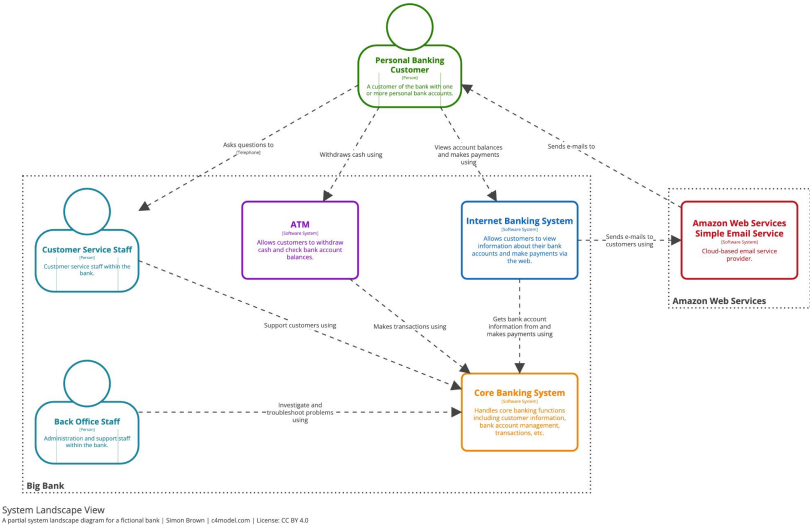
The system context diagram for a fictional Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

Container View: Internet Banking System

The container diagram for the Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

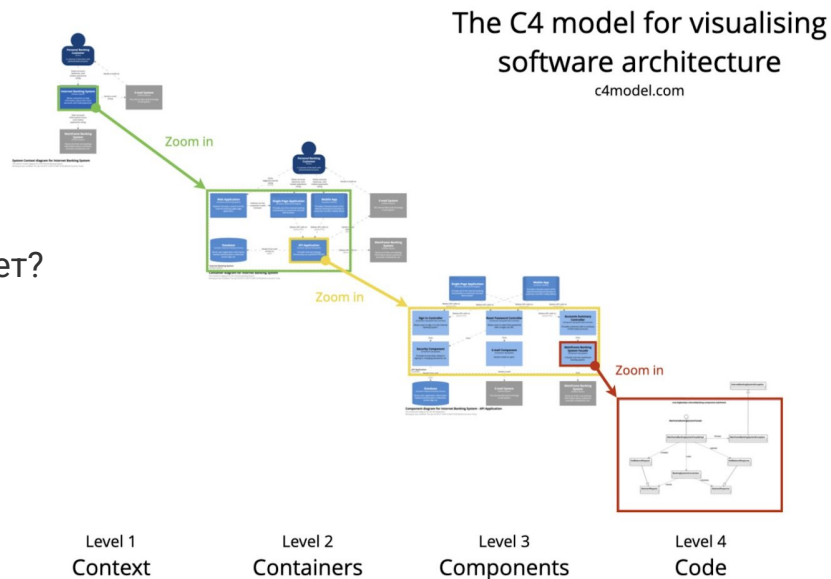
Диаграмма системного ландшафта

- Диаграмма показывает системы предприятия/организации
- Диаграмма отвечает на следующие вопросы:
 - Какие акторы есть в предприятии?
 - С какими системами взаимодействуют эти акторы?
- Основные элементы - люди и системы
- Целевая аудитория - технические и нетехнические люди



Одна модель - множество представлений

- Менеджер/Бизнес - За что мы платим деньги?
- Эксплуатация/разработка - что с чем взаимодействует?
- Разработка/эксплуатация - как это должно работать?
- Разработчик - как это работает?



С4 в контексте AI: Что меняется?

- Неопределенность: В отличие от классического ПО, поведение AI-компонентов вероятностное.
- Новые типы Контейнеров:
 - Model Registry (хранилище артефактов).
 - Feature Store (хранилище признаков).
 - Training Pipeline (процесс как часть архитектуры).
 - Inference Service (обертка над моделью).
 - Vector DB (специфичная база знаний).
- На диаграммах важно показывать потоки данных (Data Flow), так как архитектура строится вокруг данных.



Сопоставление AI компонентов на C4

- System Context (L1):
 - "AI Assistant" — как черная коробка.
 - "Data Provider" — внешняя система-источник данных.
- Container (L2):
 - Python Service (FastAPI) — стандартный контейнер.
 - ML Model — обычно не отдельный контейнер, а компонент внутри сервиса
- Component (L3):
 - Тут мы показываем внутренности: "Preprocessor", "Tokenizer", "Neural Network Class", "Post-processing logic".



Модель это контейнер или внешняя система?

SaaS

OpenAI — это серый ящик снаружи

Self-hosted

- Llama 3 в нашем Kubernetes — это контейнер
- Или даже компонент, если она деплоится одновременно с другим вашим кодом на питоне

Вопросы



если есть вопросы



если вопросов нет

C4 на примере

Внутренний Чат-бот. Бизнес-требования

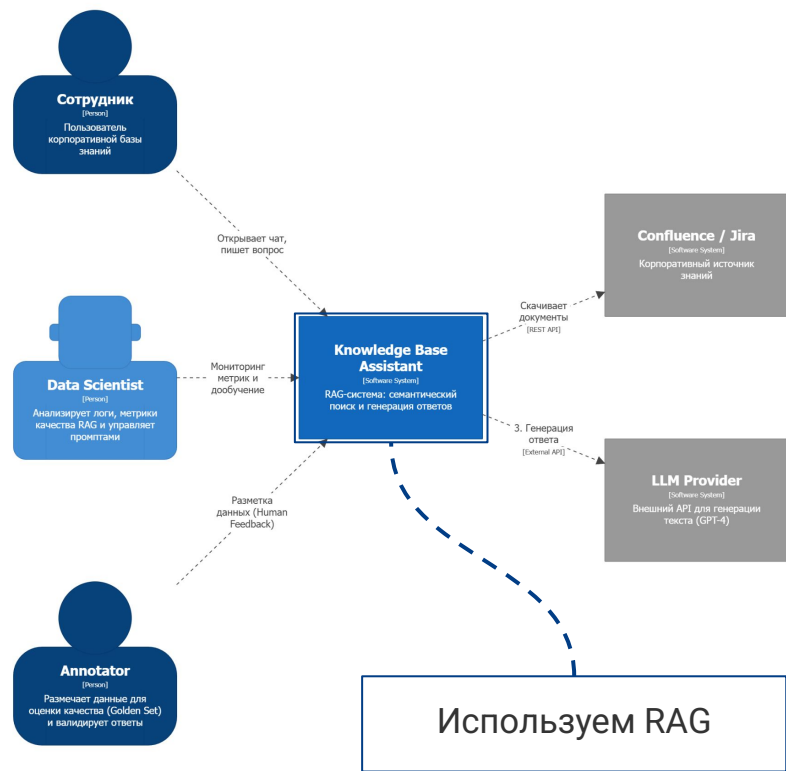
- Продукт: Внутренний Бот для операторов колл-центра. Поиск по базе знаний.
- Текущая проблема: Операторы тратят 5+ минут на поиск ответа. Клиенты висят на линии.
- Бизнес-цель: Сократить время поиска до 30 секунд.
- Жесткие ограничения (Constraints):
 - стек команды: Python (Middle), опыт в ML — начальный.



Внутренний Чат-бот. Первые архитектурные решения

Тип	Требование	Архитектурное решение
Business	Бот должен отвечать только по внутренней базе знаний (без галлюцинаций).	Использование паттерна RAG (поиск + генерация) вместо дообучения.
NFR	Время ответа пользователю должно быть < 3 секунд.	Асинхронная загрузка документов, чтобы не блокировать поиск.
Security	Персональные данные клиентов не должны уходить во внешнего провайдера	Фильтрация перс.данных перед отправкой во внешних провайдеров

Чат-бот. C4 – Level 1

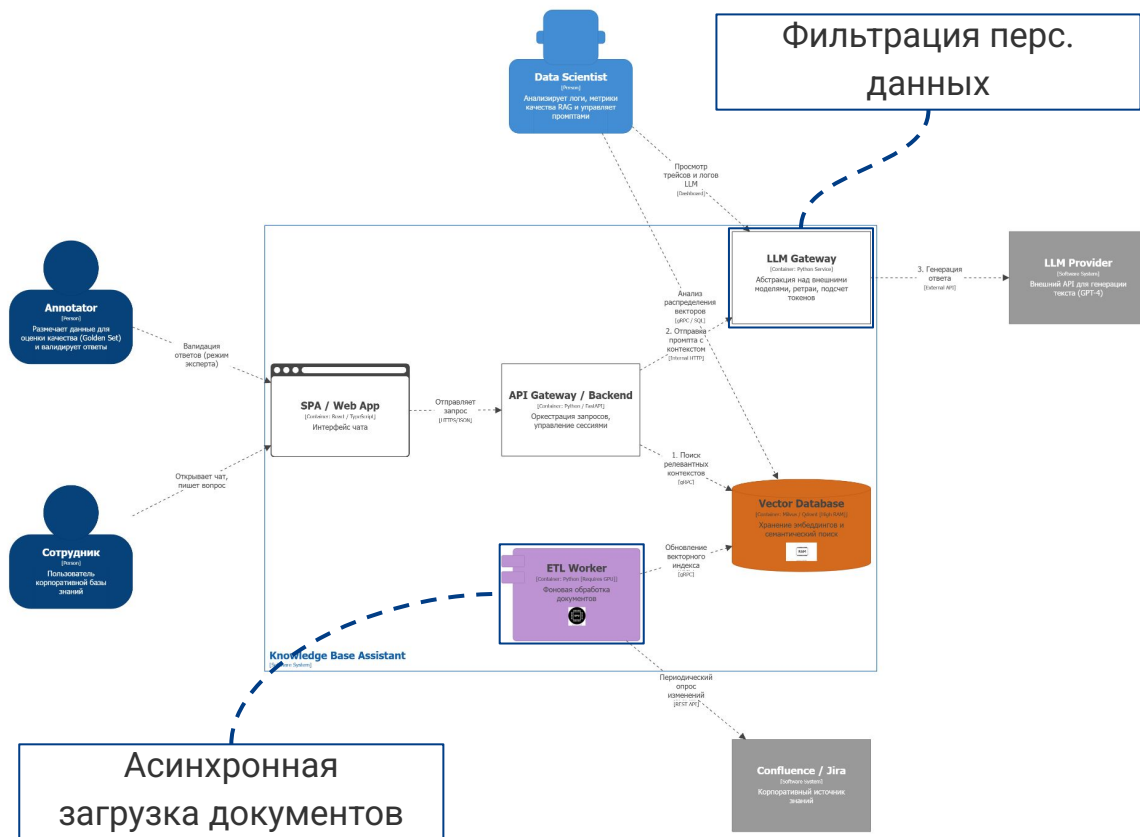


C4 Level 1: System Context
Общая схема взаимодействия пользователя и систем.
четверг, 8 января 2026 г. в 12:51 Москва, стандартное время

- В системе у нас есть и классические пользователи, и новые AI специфичные роли (Data Scientist, Annotator), которые взаимодействуют с продакшном так же часто, как и конечные клиенты. Если их не нарисовать на L1, мы забудем про их доступы при проектировании безопасности (IAM).
- Мы явно визуализируем процесс контроля качества через Аннотатора. Это архитектурное подтверждение того, что система не полностью автономна и требует валидации.
- Добавление технических ролей сразу ставит вопрос: "А как Data Scientist попадает в контур?" (VPN, Bastion host?). Это переводит разговор от абстракции к реальной сетевой безопасности.



Чат-бот. C4 – Level 2



- Контейнеры в AI неравнозначны. Пометки [GPU] и [High RAM] — это сигнал для девопсов и бюджета.
- Мы изолируем ETL Worker, чтобы тяжелая математика не "положила" API для пользователей.
- LLM Gateway — это не просто прокси. Это архитектурное место для PII-фильтрации (анонимизации данных) перед отправкой во внешний мир.

Вопросы



если есть вопросы



если вопросов нет

Инструментарий

Диаграммы vs Architecture as Code

Диаграммы

- Инструменты: Miro, Draw.io, Lucidchart, Visio.
- Подход: "Drag & Drop". Мы рисуем картинку.
- Use Case: Брейншторм, встречи с заказчиком, быстрые наброски.

АоС

- Инструменты: Structurizr DSL, PlantUML, Mermaid.
- Подход: Описываем модель текстом, движок генерирует картинку.
- Use Case: Долгосрочная документация, версионирование в Git, автогенерация.

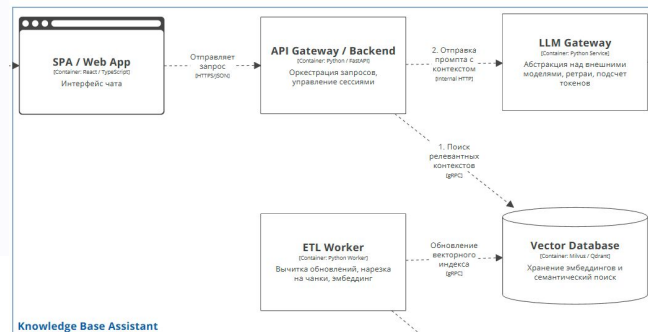
Почему картинки умирают?

- Рассинхрон: Код меняется каждый день, схема в Miro — раз в полгода.
- Сложность поддержки: Чтобы переименовать сервис, нужно найти его на 10 разных диаграммах и перепечатать текст.
- Решение "As Code": Вы меняете название в одном месте модели, и оно обновляется на всех диаграммах (Context, Container, Deployment).



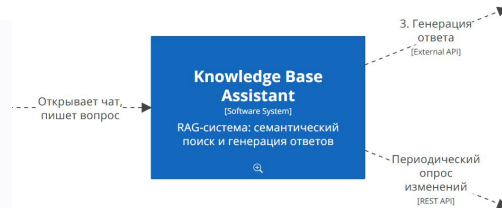
Код диаграмм из примеров (объекты)

```
model {  
  # --- Наша система (Boundary) ---  
  aiSystem = softwareSystem "Knowledge Base Assistant" "RAG-система: семантический поиск и генерация ответов" {  
    tags "Target System"  
    # Контейнеры (Детализация Level 2)  
    spa = container "SPA / Web App" "Интерфейс чата" "React / TypeScript" {  
      tags "Web Browser"  
    }  
    backend = container "API Gateway / Backend" "Оркестрация запросов, управление сессиями" "Python / FastAPI"  
    llmGateway = container "LLM Gateway" "Абстракция над внешними моделями, ретраи, подсчет токенов" "Python"  
  }  
  Service"  
  vectorDb = container "Vector Database" "Хранение эмбеддингов и семантический поиск" "Milvus / Qdrant" {  
    tags "Database"  
  }  
  etlWorker = container "ETL Worker" "Вычитка обновлений, нарезка на чанки, эмбеддинг" "Python Worker"  
}
```



Код диаграмм из примеров (связи)

```
model {  
  # --- Связи (Relationships) ---  
  
  # 1. User Interaction Flow  
  employee -> spa "Открывает чат, пишет вопрос"  
  spa -> backend "Отправляет запрос" "HTTPS/JSON"  
  backend -> vectorDb "1. Поиск релевантных контекстов" "gRPC"  
  backend -> llmGateway "2. Отправка промпта с контекстом" "Internal HTTP"  
  llmGateway -> llmProvider "3. Генерация ответа" "External API"  
  
  # 2. Data Ingestion Flow (Async)  
  etlWorker -> knowledgeBase "Периодический опрос изменений" "REST API"  
  etlWorker -> vectorDb "Обновление векторного индекса" "gRPC"  
}
```



Код диаграмм из примеров (диаграммы)

```
model {  
  # --- Диаграмма 1: System Context (Level 1) ---  
  systemContext aiSystem "ContextDiagram" {  
    include *  
    autoLayout lr  
    title "C4 Level 1: System Context"  
    description "Общая схема взаимодействия пользователя и систем."  
  }  
  # --- Диаграмма 2: Container View (Level 2) ---  
  container aiSystem "ContainerDiagram" {  
    include *  
    autoLayout lr  
    title "C4 Level 2: Containers"  
    description "Детализация: контуры Inference и ETL."  
  }  
}
```



Вопросы



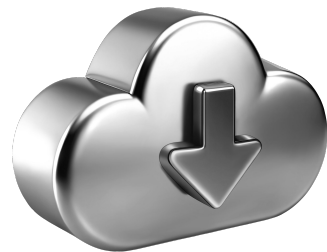
если есть вопросы



если вопросов нет

Список материалов для изучения

1. Браун, Саймон. 2022. Программная архитектура как код. М.: ДМК Пресс.
2. Ричардс, Марк, и Нил Форд. 2022. Основы программной архитектуры. Путеводитель для инженера. СПб.: Питер.
3. Мартин, Роберт. 2022. Чистая архитектура. Искусство разработки программного обеспечения. СПб.: Питер.
4. Клементс, Пол, и др. 2021. Документирование архитектуры ПО. Практическое руководство. М.: Вильямс.
5. Hohpe, Gregor. 2020. The Software Architect Elevator: Redefining the Architect's Role in the Digital Enterprise. Sebastopol, CA: O'Reilly Media.
6. Woods, Eóin. 2021. Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives. 3rd ed. Boston, MA: Addison-Wesley Professional.



**Подведём
итоги занятия**

Теперь вы сможете:

Отправьте в чат номера достигнутых целей

1. Применять модель C4 для описания архитектуры системы на разных уровнях абстракции

2. Разрабатывать диаграммы контекста, контейнеров и компонентов для AI-решений

3. Визуализировать и доносить архитектурные решения до технических и бизнес-команд

Для того чтобы выстраивать общее понятийное пространство для различных команд

Вопросы для проверки

1. Является ли ML-модель отдельным Контейнером в C4?
2. На какой диаграмме мы должны показать, что используем GPU-сервер?
3. Если у нас 5 микросервисов в одном Docker-образе (плохо, но допустим), сколько это Контейнеров в C4?



Ключевые тезисы занятия

1. Уровни абстракции — ключ к пониманию
2. Логическая архитектура $\neq$ Физическая инфраструктура
3. Architecture as Code спасает от устаревания





Отправьте в чат эмодзи, который отражает ваше настроение после занятия



Продолжите высказывание: теперь я умею/знаю...



Что планируете использовать в работе?

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

